Atividade Dirigida

API de Filmes com Node.js e SQL Server

1. Escopo do projeto

Construir uma API com Node.js que se conecta ao SQL Server e cadastra os **filmes**, com uma página HTML disponibilizada na rota padrão (`/`). Testar a API no Insomnia ou outra aplicação e documentar o processo.

2. Estrutura de pastas

```
api-filmes/
├── public/
│   ├── index.html
│   └── estilo.css
├── src/
│   ├── config/
│   │   └── bdConnect.js
│   ├── controllers/
│   │   └── filmeController.js
│   ├── routes/
│   │   └── filmeRoutes.js
│   └── app.js
├── package.json
├── package-lock.json
└── server.js
```

3. Roteiro de desenvolvimento

Etapa 1: Criação das tabelas no banco de dados SQL Server

Crie as tabelas usando o esquema **`dbFilmes`:**

Tabela `Categorias`

Coluna	Tipo	Restrições
id	INT	IDENTITY(1,1)
nome	VARCHAR(50)	NOT NULL, UNIQUE

Tabela `Filmes`

Coluna	Tipo	Restrições
id	INT	IDENTITY(1,1)
titulo	VARCHAR(150)	NOT NULL
categoria_id	INT	NOT NULL
diretor	VARCHAR(100)	NOT NULL
ano_lancamento	INT	NULL
duracao_minutos	INT	NULL

Depois, insira **pelo menos 5 categorias** (Drama, Comédia, Suspense, Ação, Terror...) e **pelo menos 8 filmes**. Salve tudo em um arquivo `script.sql` e entregue junto com o projeto.

Etapa 2: Iniciar o projeto

Seguir os passos abaixo para iniciar o projeto e instalar as dependências:

```
mkdir api-filmes && cd api-filmes
npm init -y
npm install express mssql cors dotenv
npm install --save-dev nodemon
```

No arquivo `package.json`, faça os ajustes abaixo:

```
{
  "name": "api-filmes",
  "version": "1.0.0",
  "type": "module",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

Crie também um arquivo `.env` na raiz com as informações da porta e da string de conexão com o banco de dados:

```
PORTA=8080
CONNECTION_STRING=Server=regulus.cotuca.unicamp.br;Database=BDXXXXX;User
Id=BDXXXXX;Password=BDXXXXX;TrustServerCertificate=True
```

Etapa 3: Conexão com o banco, src/config/dbConnect.js

Orientações:

1. Importe `mssql` e carregue o `dotenv`.
2. Crie **uma função para fazer a conexão com o servidor de banco de dados**.
3. Exporte uma função que retorna a conexão estabelecida.

```
import mssql from 'mssql';
import 'dotenv/config';

const stringSQL = process.env.CONNECTION_STRING;

// conectar no BD
async function conectaBD() {
    // corpo da função para conectar ao servidor de banco de dados


}

export default conectaBD;
```

Etapa 4: Aplicação Express, src/app.js

Esta etapa contém a configuração mais importante: **servir o `index.html` na rota padrão**.

Orientações:

1. Importe `express`, `cors` e `path`.
2. Habilite `express.json()` para ler o corpo das requisições.
3. Use `express.static` apontando para a pasta `public`. Como `app.js` está dentro de `src/`, é preciso **subir um nível** com `..`.
4. Crie a rota `GET /` que envia o `index.html` com `res.sendFile`, que exige **caminho absoluto**.
5. Registre as rotas da API com o prefixo `/api`.

```
import express from "express";
import bodyParser from 'body-parser';
import routes from "./routes/index.js";
import path from 'path';
import { fileURLToPath } from 'url';
import cors from 'cors';
import filmeRoutes from './routes/filmeRoutes.js';

// Em ES Modules, __dirname NÃO existe: é preciso recriá-lo
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();

// Adds headers: Access-Control-Allow-Origin: *
app.use(cors());

// Configura o middleware para entender requisições no formato JSON
app.use(bodyParser.json());

// Arquivos estáticos (style.css, imagens, etc.)
app.use(express.static(path.join(__dirname, '..', 'public')));

// Rota default: carrega o index.html
app.get('/', (req, res) => {
res.sendFile(path.join(__dirname, '..', 'public', 'index.html'));
});

routes(app);

export default app;
```

Atenção: *`__dirname` aponta para a pasta do arquivo atual (`src/`). Por isso `path.join(__dirname, '..', 'public')` chega à pasta `public` da raiz. Se o caminho estiver errado, o navegador exibirá `Cannot GET /` ou um erro de `ENOENT`.*

No `index.html`, o link para o CSS deve ser `<link rel="stylesheet" href="style.css">` (sem `public/` no caminho). O `express.static` já trata a pasta `public` como raiz do site.

Etapa 5: O servidor server.js

Importe `app` de `./src/app`, leia **`PORTA`** do `.env` (padrão 8080) e chame `app.listen`, exibindo uma mensagem no console. Se quiser, teste a conexão com o banco antes de subir o servidor.

Etapa 6: Rotas, src/routes/filmeRoutes.js

Use `express.Router()` e defina as rotas abaixo, associando cada uma a uma função do controller:

Método	Rota	Descrição
GET	`/categorias`	Lista todas as categorias
POST	`/categorias`	Cadastra uma categoria
GET	`/filmes`	Lista filmes (com o nome da categoria, via JOIN)
GET	`/filmes/:id`	Busca um filme pelo id
POST	`/filmes`	Cadastra um filme
PUT	`/filmes/:id`	Atualiza um filme
DELETE	`/filmes/:id`	Remove um filme

Etapa 7: Model, src/models/Filme.js e src/models/Categoria.js

Criar a classe Filme e Categoria para representar as respectivas entidades com os respectivos atributos (consultar tabela do BD) e métodos (consultar tabela de rotas acima).

Etapa 8: Controller, src/controllers/filmeController.js e src/controllers/categoriaController.js

Implemente uma função assíncrona para cada rota. Orientações obrigatórias:

- Use `try/catch` em todas as funções e responda com `res.status(500).json({ erro: '...' })` em caso de falha.
- Use **parâmetros nomeados** (`request.input('titulo', sql.VarChar, titulo)`). **Não concatene valores na string SQL**, para evitar SQL Injection.
- No `GET /filmes`, faça `INNER JOIN` com `Categorias` para retornar `categoria` (nome) junto com o filme.
- Valide os campos obrigatórios e retorne **400** se faltarem.
- Retorne os códigos HTTP corretos: **200** (OK), **201** (criado), **400** (requisição inválida), **404** (não encontrado), **500**(erro interno).

```
import curso from "../models/Filme.js";

class FilmeController {

    static async listarFilmes(req, res) {
        try {

           res.status(200).json(result.recordset);
        } catch (erro) {
            res.status(500).json({ message: `${erro.message} - falha na
requisição` });
        }
    }

    // CONTINUA...
}

export default FilmeController;
```

Etapa 9: Página inicial

No `index.html`, crie uma página simples que:

- Tenha um título e um menu ou instruções de uso da API;
- Use `fetch('/api/filmes')` para listar os filmes em uma tabela;
- Use o `style.css` para estilizar (cores, tabela, fonte).

Etapa 10: Testes no Postman ou Insomnia

Crie uma **coleção** chamada `API Filmes` e teste **todas** as rotas. Para cada teste, tire um print mostrando a requisição (método, URL, corpo) e a resposta (status e JSON). Inclua obrigatoriamente os cenários abaixo.

#	Cenário	Resultado esperado
1	GET `/api/categorias`	200 e lista de categorias
2	GET `/api/filmes`	200 com o nome da categoria
3	GET `/api/filmes/9999`	404

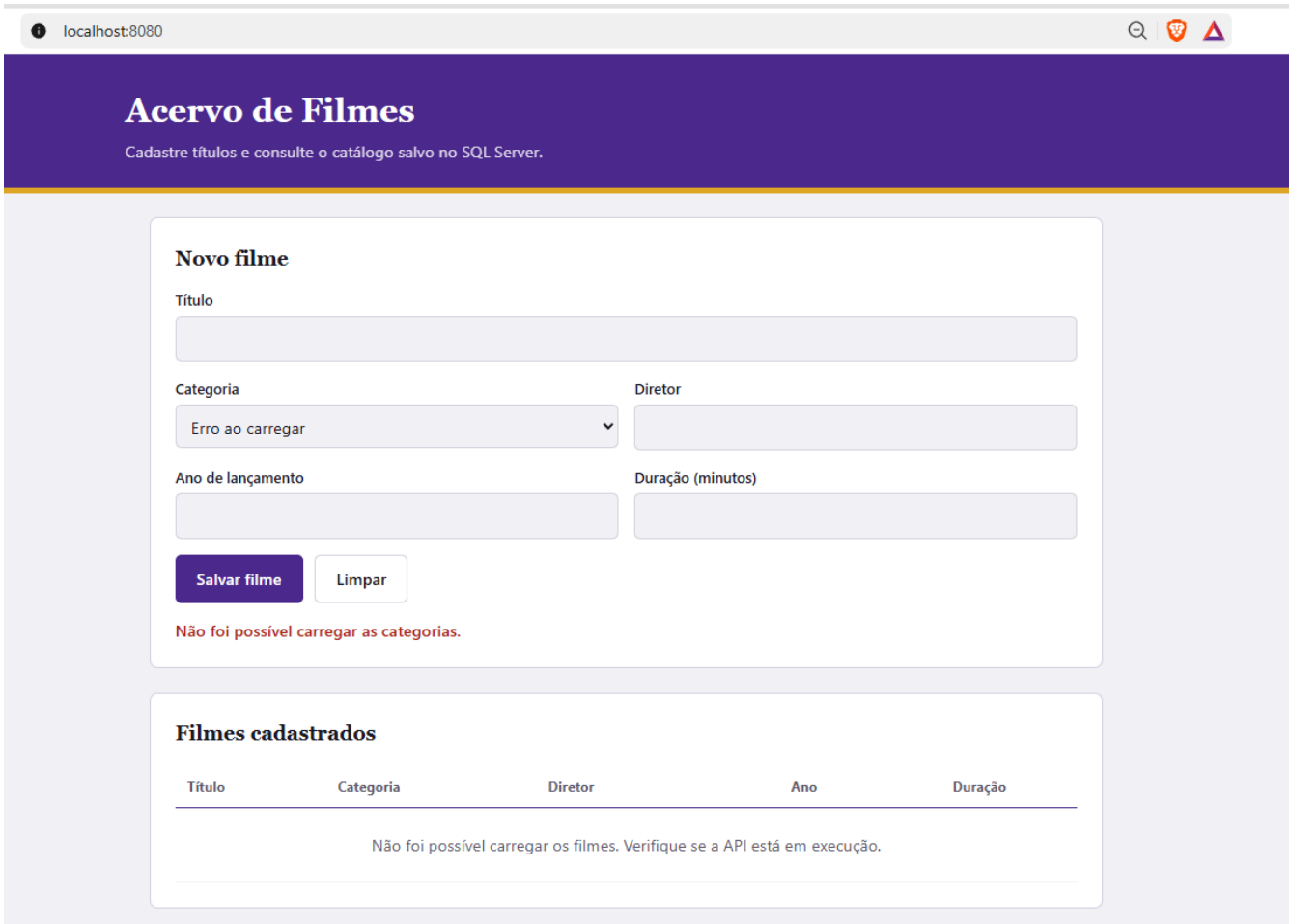
4	POST `/api/filmes` válido	201
5	PUT `/api/filmes/:id`	200
6	DELETE `/api/filmes/:id`	200 ou 204

Etapa 11: Página inicial do projeto

Criar o arquivo `index.html` e `estilo.css` na pasta `public`. Exibir um formulário para inserção dos dados para cadastro de filmes e abaixo uma tabela com a lista dos filmes cadastrados no banco de dados.

No formulário, no campo **categoria**, preencher automaticamente com os nomes das categorias cadastradas no banco de dados em um elemento **SELECT**. O `value` deve ser o ID da categoria.

Exemplo:



5. Entregar

1. Pasta do projeto compactada **sem** `node_modules` e **sem** `.env` (inclua um `.env.example`);
2. Arquivo `script.sql`;
3. Coleção exportada do Postman/Insomnia **ou** documento com os prints dos 10 testes;
4. Respostas das questões da Seção 6.

6. Questões

1. Na estrutura do projeto, qual é a função do arquivo `server.js`?

a) Definir as rotas da API
b) Iniciar o servidor HTTP na porta configurada, importando a aplicação de `src/app.js`
c) Criar as tabelas no banco de dados
d) Armazenar as dependências do projeto

2. A linha `app.use(express.static(path.join(__dirname, '..', 'public')))` tem como efeito:

a) Criar a pasta `public` automaticamente
b) Permitir que os arquivos da pasta `public` (CSS, HTML, imagens) sejam acessados diretamente pelo navegador
c) Impedir o acesso aos arquivos da pasta `public`
d) Converter os arquivos HTML em JSON

3. Por que, em `app.js`, foi necessário usar `'..'` em `path.join(__dirname, '..', 'public')`?

a) Porque `public` é uma pasta oculta
b) Porque o Express exige sempre dois pontos no caminho
c) Porque `app.js` está dentro de `src/` e a pasta `public` está um nível acima, na raiz do projeto
d) Porque `__dirname` retorna a raiz do projeto

4. Explique o fluxo completo de uma requisição `GET /api/filmes`, desde o momento em que ela chega ao servidor até a resposta ser devolvida ao Insomnia. Cite o papel de `server.js`, `app.js`, `filmeRoutes.js`, `filmeController.js` e `dbConnect.js`.

7. Desafio

Pesquise sobre o uso de um **Pool de Conexões** (*Connection Pool*) para fazer a conexão com o banco de dados ao invés de fazer uma conexão para cada requisição da API. Explique o que entendeu com exemplos de código.